

A Regulatory-Compliant Apiary Management and E-Commerce System: Honigmanufaktur Lindenhof

Hafsa Iddris
Graduate School
Hof University
Hof, Germany
hiddris@hof-university.de

Emil Shkurti
Graduate School
Hof University
Hof, Germany
eshkurti@hof-university.de

Fahimeh Tajarrod
Graduate School
Hof University
Hof, Germany
ftajarrod@hof-university.de

Abstract- Honigmanufaktur Lindenhof is a semi-professional honey producer managing approximately 120 colonies across six apiary stands in Landkreis Hof, Bavaria. Previously relying entirely on paper-based workflows - handwritten Stockkarten filed inside individual hive boxes and a manually maintained Bestandsbuch treatment ledger - the operation faced both systemic compliance risk under the German Animal Medical Products Act (TAMG 2022) and the inability to execute a product recall without days of manual cross-referencing. This paper describes a web application that replaces those paper workflows with a unified digital platform. The system is implemented on the Yii2 PHP framework as a Monolithic Layered MVC application using the Advanced multi-application template, separating a public customer shop (frontend) from a staff administration panel (backend) while sharing a common domain layer and a single MariaDB database. Business logic is organized into three bounded modules - Production Management, Compliance, and Ecommerce - held in the shared domain layer. Role-based access control with five roles enforces least privilege at the controller level. A five-condition Release Gate automatically evaluates statutory compliance before any batch may be approved for sale, ensuring no honey from a colony within an active Wartezeit enters commerce. The system is verified by a Codeception test suite that deliberately includes failure cases, and validated against all functional requirements through end-to-end user journeys.

Keywords- *apiary management, beekeeping compliance, veterinary traceability, agricultural information systems, role-based access control, ecommerce, Yii2*

I. INTRODUCTION

Honigmanufaktur Lindenhof is a semi-professional honey producer based in Hof in the Upper Franconia region of Bavaria, Germany. The enterprise employs approximately fifteen people distributed across four operational areas: three field beekeepers who manage the colonies at six remote apiary stands, one head beekeeper responsible for production decisions and compliance oversight at the central facility, one administrator who maintains the colony register and customer records, and additional office and logistics staff. The company manages between 110 and 130 colonies that are migrated seasonally to capture specific regional floral blooms: rapeseed in April and May, mixed wildflower and forest stands from

June to August, and heather in late summer. Extracted honey is processed, packaged, and sold through two channels: a direct-to-consumer online shop and a small number of wholesale accounts supplying local bakeries and a regional delicatessen.

A. Problem Statement

The operation has historically managed all record-keeping through physical, paper-based systems. Field apiculturalists record colony health observations, queen status, and disease indicators on weather-resistant Stockkarten kept inside individual hive boxes. Veterinary treatment records are transcribed onto a separate paper Bestandsbuch at the central facility, as mandated by EU Regulation 2019/6 Article 108(2) [1] and the German Animal Medical Products Act (TAMG 2022) [2]. These records must be retained for a minimum of five years and produced on demand to the Veterinärämter.

This reliance on decentralized paper records introduces three critical vulnerabilities. First, treatment compliance data is physically isolated at remote apiary stands. A field worker who applies a Varroa treatment and records it on a Stockkarte has produced a compliance record that is trapped in the field. If honey from that colony is subsequently extracted before the paper record is manually transferred and cross-referenced against the statutory withdrawal period (Wartezeit), the enterprise commits a violation of TAMG 2022. Second, there is no connection between the treatment ledger and the online shop. In a recall scenario, identifying which customers received honey from a specific colony or batch requires manually cross-referencing paper lot labels against order records, a process that is both error-prone and legally inadequate. Third, the withdrawal period after a treatment-during which honey from the treated colony cannot be harvested or sold-rests entirely in the head beekeeper's memory and informal notes.

B. Pain and Gain

The pain is concrete and quantifiable. Germany is home to approximately 900,000 registered beekeepers keeping some 1.2 million colonies, making it the largest beekeeping nation in Europe [3]. Despite this scale, the domestic honey harvest satisfies only about one-third of national consumption, with

imports making up the deficit [4]. For a migratory operation like Lindenhof, maintaining statutory compliance is both a regulatory obligation and a commercial prerequisite: honey from a colony in an active Wartezeit cannot legally be sold, and a single documentation failure can require a full product recall. The current paper-based process provides no deterministic prevention of such failures.

The expected gain is staged over time. In the first year, digitising the field workflow removes the physical isolation of compliance data: a treatment entered at the stand immediately computes the Wartezeit expiry, blocks the colony from contributing to any batch whose harvest date falls before that expiry, and records the EU-required Bestandsbuch fields in the same operation. The head beekeeper’s release decision is supported by an automatically evaluated five-condition Release Gate rather than manual cross-referencing. Customer orders permanently record the batch lot number, reducing recall tracing from days to seconds. In the second year, the platform can be extended to support formal authority notifications to the Veterinäramt, pesticide consent management with neighbouring farmers, and laboratory result integration, building on the extension points already part of the system design.

II. STATE OF THE ART

The context for this project sits at the intersection of a growing organic food market, tightening veterinary traceability legislation, and an underserved software niche in German commercial apiculture.

A. Market and Regulatory Context

The German organic food market reached a turnover of approximately 17 billion euros in 2024, with organic products now purchased by around 97 percent of German households [5]. Honey, as a premium natural product with strong regional provenance, sits directly in this growth segment. At the same time, EU Regulation 2019/6, which took effect in January 2022 and is implemented in Germany through TAMG 2022, significantly strengthened the mandatory documentation obligations for veterinary medicinal products used in livestock farming, including beekeeping. Article 108(2) of that regulation specifies eleven mandatory fields that must appear in the Bestandsbuch for every treatment event, and requires their retention for five years [1]. These obligations create a compliance burden that paper-based systems satisfy poorly.

B. Existing Systems and Approaches

Several categories of software partially address the domain. General-purpose farm management platforms, such as AgrarOffice and agroVet, provide treatment logging and animal health records for conventional livestock, but are not structured around the specific entity model of beekeeping: the Apiary Stand as the registered compliance unit, the Colony as the individual biological asset, and the Stockkarte as the per-colony chronological record. Beekeeping-specific tools such as BeeLogic and ImkerDB offer hive inspection logging and colony management for hobbyist and semi-professional use,

but do not implement the Bestandsbuch mandatory fields under EU Regulation 2019/6 and have no connection to product sales. Commercial honey ecommerce uses general-purpose platforms (Shopify, WooCommerce) that have no awareness of lot traceability or withdrawal periods. No existing product connects treatment records at the apiary stand to batch release decisions at the facility and to customer order records in the shop. The gap is both functional and architectural.

C. Architectural Foundations

The application is built on well-established software-engineering foundations. The system follows a Monolithic Layered MVC architectural style, realised through the Yii2 Advanced multi-application template [6]. This partitions the application into two independent web applications—a public customer shop (frontend) and a staff administration panel (backend)—that share a common domain layer and a single database. Responsibility is separated across three explicit tiers: a Presentation layer (Controllers and Views within each application), a Domain layer (shared ActiveRecord models in common/ that encapsulate all business logic, validation, and compliance rules), and a Data layer (MariaDB persisted through Yii2’s Active Record ORM). Infrastructure concerns such as the database engine, export format, and mail delivery are handled by framework-provided components beneath the domain layer, keeping the domain models free of external dependencies. Authorisation is handled through role-based access control, in which permissions are attached to roles rather than directly to individual users, keeping access decisions consistent and maintainable as the system grows [7]. Modern PHP model-view-controller frameworks provide object-relational mapping, input validation, and schema migration support [8], and the use of migrations to evolve a database schema in a controlled and repeatable way is a recognised software-engineering practice [9]. These foundations are combined in the Yii2 PHP framework, which provides all three capabilities in a single, well-documented package and is the standard framework for this project [6].

III. SPECIFICATION

This chapter specifies what the system must do and the qualities it must have. Five kinds of actor interact with it. A **Guest** is an unauthenticated visitor who can browse the public shop. A **Customer** is a registered buyer who can place orders and track their own purchases. A **Field Beekeeper** records inspections, treatments, and harvests at apiary stands. A **Head Beekeeper** oversees all production, releases batches, publishes products, and manages the operations dashboard. An **Administrator** manages system configuration: the apiary/colony register, user accounts, customer records, and the company profile.

A. User Stories

The system requirements were derived from structured user story workshops with the Lindenhof team, producing 12 user stories distributed across the three operational modules, as shown in Table I. Production Management accounts for five

stories (US-01 through US-05), covering the complete field lifecycle from apiary configuration and colony registration through inspection, treatment, and batch production. The Compliance module contributes three stories (US-06 through US-08), addressing release gate evaluation, regulatory export generation, and disease flag management with recall traceability. The Ecommerce module provides the remaining four stories (US-09 through US-12), spanning the public catalogue and customer registration, order placement and tracking, product publishing from released batches, and administrative oversight of customers and the operations dashboard.

TABLE I. USER STORIES

ID	Title	Role	US
US-01	Configure system and manage apiary registry	Administrator	US-PM-00, US-PM-01, US-PM-02
US-02	Log colony inspection	Field Beekeeper	US-PM-03
US-03	Record veterinary treatment (individual and bulk)	Field Beekeeper	US-PM-04, US-PM-08
US-04	Record harvest and complete batch production	Field / Head Beekeeper	US-PM-05, US-PM-06
US-05	View colony Stockkarte	Field Beekeeper	US-PM-07
US-06	Evaluate release gate and release batch	Head Beekeeper	US-CO-01, US-CO-02
US-07	Generate compliance exports (Bestandsbuch and Stockkarte)	Administrator	US-CO-03, US-CO-04
US-08	Manage disease concerns and recall traceability	Head Beekeeper	US-CO-05, US-CO-06
US-09	Browse catalogue and register customer account	Guest	US-EC-01, US-EC-02
US-10	Place and track orders	Customer	US-EC-03, US-EC-04
US-11	Publish products and process order fulfilment	Head Beekeeper	US-EC-05, US-EC-06
US-12	Manage customers and monitor operations dashboard	Administrator	US-EC-07, US-EC-08

B. Functional Requirements

Table II lists the 23 functional requirements derived from the user stories. All are in scope for this implementation. The scope column in Table III maps each requirement to its user story origin and to the verification means described in Chapter VI.

TABLE II. FUNCTIONAL REQUIREMENTS

ID	Requirement (summary)	US
FR 1	Maintain company profile (Tierhalter identity) for compliance export headers	US-01
FR 2	Manage Apiary Stand register with GPS, Landkreis, and authority registration number	US-01

FR 3	Manage Colony register with unique IDs, queen year, status, movement events, and annual treatment compliance flags	US-01
FR 4	Record structured inspection logs per colony with date, inspector, weather, brood, and disease indicators	US-02
FR 5	Record veterinary treatments with all EU Reg. 2019/6 Art. 108(2) fields; autofill from approved-product reference	US-03
FR 6	Automatically calculate and store Wartezeit expiry; block treated colony from batches harvested before expiry	US-03
FR 7	Record harvest events linking source colonies and stands to a Batch	US-04
FR 8	Record Batch production details including variety, water content, HMF, lot number, HACCP confirmation, and container validation	US-04
FR9	Five-condition Release Gate: Wartezeit, water content, label fields, disease flag, HACCP	US-06
FR1 0	Generate Bestandsbuch export (CSV and PDF) per stand or all stands over a date range	US-07
FR1 1	Generate Stockkarte export per colony or per stand	US-07
FR1 2	Recall search by colony identifier or batch lot number returning all affected batches and orders	US-08
FR1 3	Public ecommerce storefront with catalogue, cart, checkout, and order tracking	US-09, US-10
FR1 4	Synchronise released batches to ecommerce products with inherited traceability fields	US-11
FR1 5	Order lines permanently record batch lot number at time of purchase	US-10
FR1 6	Internal order fulfilment lifecycle with four stages and operator audit trail	US-11
FR1 7	CRM: customer and company records with wholesale designation and communication notes	US-12
FR1 8	Operations dashboard: open orders, low-stock alerts, active Wartezeiten, batches pending release	US-12
FR1 9	RBAC: five roles with controller-level permission enforcement	All
FR2 0	Creation timestamps and submitting user identity on all compliance-relevant records	All
FR2 1	Bulk treatment entry: one form entry creates one treatment record per selected colony transactionally	US-03
FR2 2	Batch packaging validation: unit count ceiling, HMF cap, chronological date enforcement	US-04
FR2 3	Batch allocation tracking: prevent over-allocation across sibling products	US-11

C. Non-Functional Requirements

TABLE III. NON-FUNCTIONAL REQUIREMENTS

ID	Requirement
NFR 1	Security: RBAC at controller level, password hashing, encrypted transport

NFR 2	Data integrity: transactional stock updates; referential integrity across colony-to-order chain
NFR 3	Auditability: creation timestamps and actor identity on all compliance-relevant entities
NFR 4	Regulatory alignment: Bestandsbuch fields match EU Reg. 2019/6 Art. 108(2); water content limits match HonigV
NFR 5	Usability: internal workflows optimised for desktop and tablet; field beekeeper completes treatment record in under two minutes
NFR 6	Performance: dashboard and search queries complete within practical operational thresholds on the shared university server
NFR 7	Extensibility: modular Yii2 design allows future integrations without structural change to the shared data model
NFR 8	Export quality: Bestandsbuch and Stockkarte exports are complete, consistent, and unmodified from source records

D. Requirements Traceability

Table IV maps each functional requirement to the user story that motivates it and to how it is verified, so that every requirement is accounted for.

TABLE IV. REQUIREMENTS TRACEABILITY MATRIX

FR1–FR3 (*US-01; AC-PM-00–AC-PM-02*): Functional test (HarvestToReleaseCest: adminCreatesApiaryStand, adminRegistersColonyAtStand) for FR2–FR3; manual walkthrough for FR1.

FR4 (*US-02; AC-PM-03.1–05*): Manual walkthrough.

FR5–FR6, FR21 (*US-03; AC-PM-04.1–09, AC-PM-08.1–06*): Functional test (HarvestToReleaseCest: treatmentRecordedWithWartezeitCalculated) for FR5–FR6; manual walkthrough for FR21.

FR7–FR8, FR22 (*US-04; AC-PM-05.1–04, AC-PM-06.1–08*): Functional test (HarvestToReleaseCest: harvestCreatesBatchInPendingRelease) for FR7–FR8; manual walkthrough for FR22.

FR9 (*US-06; AC-CO-01.1–06, AC-CO-02.1–06*): Functional test (HarvestToReleaseCest: releaseGateBlockedByActiveWartezeit, releaseSucceedsWhenAllChecksPassed).

FR10–FR11 (*US-07; AC-CO-03.1–06, AC-CO-04.1–06*): Functional test (HarvestToReleaseCest: bestandsbuchExportProducesCSVForStand) for FR10; manual walkthrough for FR11.

FR12 (*US-08; AC-CO-05.1–07, AC-CO-06.1–05*): Functional test (HarvestToReleaseCest: recallTraceFindsOrdersForLotNumber, diseaseFlagPutsReleasedBatchUnderReviewAndUnpublishesProduct).

FR13, FR15 (*US-09, US-10; AC-EC-01–AC-EC-04*): Functional test (ShopCheckoutCest: productDetailShowsLotTraceability, checkoutDecrementsStockAndSnapshotsLotNumber, checkoutRequiresDeliveryAddress).

FR14, FR23 (*US-11; AC-EC-05.1–08*): Functional test (HarvestToReleaseCest:

headBeekeeperCreatesProductFromReleasedBatch) for FR14; manual walkthrough for FR23.

FR16 (*US-11; AC-EC-06.1–05*): Manual walkthrough.

FR17–FR18 (*US-12; AC-EC-07.1–05, AC-EC-08.1–05*):

Functional test (ShopCheckoutCest: wholesaleMinimumEnforced) for FR17 wholesale designation; manual walkthrough for remainder.

FR19–FR20 (*all user stories; all acceptance-criteria sets*):

Functional test (HarvestToReleaseCest: releaseSucceedsWhenAllChecksPassed, asserting released_by/released_at) for FR20; manual walkthrough for FR19.

E. Use Cases

The following paragraphs present the derived use cases for each of the three system modules. Each use case is keyed to its module prefix (PM, CO, EC), identifies the primary triggering actor and any secondary actors involved, states the precondition that must hold before the use case begins, and describes the main success outcome. The actor definitions match those introduced in the module overview at the start of this chapter: Administrator, Field Beekeeper, Head Beekeeper, Customer, and Guest. The per-module use case diagrams are given in Fig. 1, Fig. 2, and Fig. 3.

Production Management Use cases (UC-PM-01–UC-PM-07).

UC-PM-01: Configure System & Manage Apiary Registry.

Primary Actor: Administrator. **Precondition:** Administrator is authenticated. **Main success outcome:** Company profile saved; apiary stand created or updated with GPS, Landkreis, and authority registration number.

UC-PM-02: Manage Colony Register:

Primary actor: Administrator. **Precondition:** Apiary stand exists. **Main success outcome:** Colony registered with unique code, queen year, and stand assignment; movement recorded when colony relocated.

UC-PM-03: Log Colony Inspection:

Primary actor: Field Beekeeper. **Precondition:** Colony is active; beekeeper authenticated. **Main success outcome:** Inspection records saved with date, weather, brood score, queen sighted flag, disease indicators, and inspector identity.

UC-PM-04: Record Veterinary Treatment (Individual & Bulk):

Primary actor: Field Beekeeper. **Precondition:** Colony/stand exists; product references available. **Main success outcome:** Treatment record(s) saved with all EU Reg. 2019/6 Art. 108(2) fields; Wartezeit expiry auto calculated and colony blocked from harvest until expiry.

UC-PM-05: Record Harvest & Batch Production.

Primary actor: Field Beekeeper / Head Beekeeper. **Precondition:** Source colonies are Wartezeit-cleared and disease-free. **Main success outcome:** Harvest linked to source colonies; lot number generated; batch placed in Pending Release status.

UC-PM-06: Complete Batch Details & Validation:

Primary actor: Head Beekeeper. **Precondition:** Batch in Pending Release. **Main success outcome:** Honey variety, water content, HMF, HACCP confirmation, and packaging details recorded; batch validation rules enforced.

UC-PM-07: View Colony Stockkarte.

Primary actor: Field Beekeeper / Head Beekeeper. **Precondition:** Colony exists. **Main success outcome:** Chronological record of inspections, treatments, harvests, and feedings displayed for the selected colony.

These production management use cases and their actors are summarized in Fig. 1.

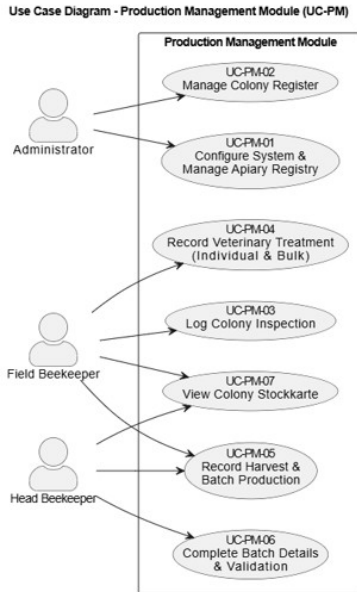


Fig. 1. Production management use case diagram (UC-PM-01-UC-PM-07): apiary registry configuration, colony register, inspections, treatments, harvests, batch production, and Stockkarte viewing.

Compliance use cases (UC-CO-01-UC-CO-06).

UC-CO-01, Evaluate Release Gate (5-condition check).

Primary actor: Head Beekeeper. **Precondition:** Batch in Pending Release or Review Required. **Main success outcome:** Five gate checks evaluated live: Wartezeit cleared, water content within HonigV limits, label fields complete, no disease flag on source colonies, HACCP confirmed.

UC-CO-02, Release Batch to Ecommerce.

Primary actor: Head Beekeeper. **Precondition:** All five gate conditions pass. **Main success outcome:** Batch status set to Released; releasing user identity and timestamp recorded; batch becomes eligible for product publication.

UC-CO-03, Generate Bestandsbuch Export (CSV & PDF).

Primary actor: Administrator. **Precondition:** Treatments exist for selected stand and date range. **Main success outcome:** EU Reg. 2019/6 Art. 108(2) compliant export produced; Tierhalter name and address in document header; downloadable as CSV or PDF.

UC-CO-04, Generate Stockkarte Export (CSV & PDF).

Primary actor: Administrator. **Precondition:** Colony exists at selected stands. **Main success outcome:** Per-colony chronological record exported with all inspection, treatment, harvest, and feeding entries and submitting user identity.

UC-CO-05, Manage Disease Concern Flag.

Primary actor: Head Beekeeper. **Precondition:** Colony exists, and concern has been identified. **Main success outcome:** Disease flag set with mandatory descriptive note; released batches from that colony moved to Review Required; flag cleared with resolution note after investigation.

UC-CO-06, Perform Recall Trace (colony or lot number).

Primary actor: Head Beekeeper. **Precondition:** Batch lot number or colony code known. **Main success outcome:** Trace returns all

affected batches and every order line item across all fulfilment stages; customer names, order dates, and quantities surfaced.

These compliance use cases and their actors are summarized in Fig. 2.

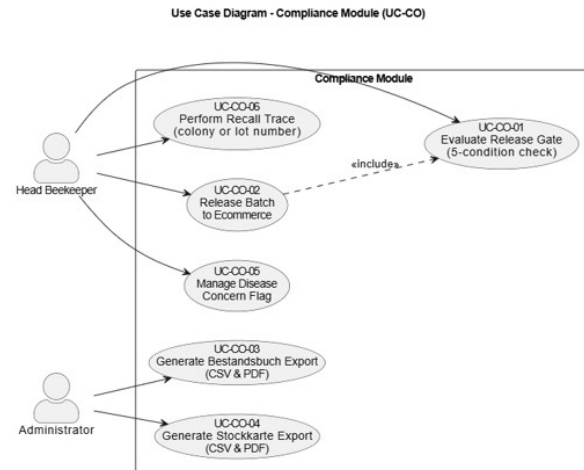


Fig. 2. Compliance use case diagram (UC-CO-01-UC-CO-06): Release Gate evaluation, batch release, Bestandsbuch and Stockkarte exports, disease concern management, and recall tracing.

Ecommerce use cases (UC-EC-01-UC-EC-08).

UC-EC-01, Browse Honey Catalogue.

Primary actor: Guest/Customer. **Precondition:** Products published with stock available. **Main success outcome:** Published products with provenance panel (lot number, variety, harvest date, water content, origin) displayed; wholesale customers see their negotiated price.

UC-EC-02, Register Customer Account.

Primary actor: Guest. **Precondition:** Valid email address not yet registered. **Main success outcome:** Account created and activated immediately; Customer CRM record created. Email verification is deferred to V2.

UC-EC-03, Add to Cart & Place Order.

Primary actor: Customer. **Precondition:** Account active; at least one product in stock. **Main success outcome:** Cart assembled; delivery address confirmed; order placed with stock decremented; order line items permanently record source batch lot number.

UC-EC-04, Track Order Status.

Primary actor: Customer. **Precondition:** Customer has placed at least one order. **Main success outcome:** Order history shown; current fulfilment stage and stage-change timestamps visible; only the authenticated customer's own orders are accessible.

UC-EC-05, Manage Products & Publish from Released Batches.

Primary actor: Head Beekeeper. **Precondition:** Batch has been released and has units available. **Main success outcome:** Product created from released batch; provenance fields inherited automatically; product published to shop or unpublished; allocation ceiling enforced to prevent over-allocation.

UC-EC-06, Process Order Fulfilment (4 stages).

Primary actor: Head Beekeeper/Administrator. **Precondition:** Order received in the system. **Main success outcome:** Order advanced through Received → Packed → Shipped → Delivered; each transition logged with operator identity and optional note; order

cancellable with automatic stock restoration while still in warehouse.

UC-EC-07, Manage Customers & Companies (CRM).

Primary actor: Administrator. **Precondition:** Administrator is authenticated. **Main success outcome:** Customer and company records created, updated, or deactivated; wholesale designation and minimum order quantity set; communication notes recorded with timestamp.

UC-EC-08, Monitor Operations Dashboard.

Primary actor: Administrator / Head Beekeeper. **Precondition:** Authenticated staff session. **Main success outcome:** Dashboard surfaces open orders, low-stock alerts, active Wartezeiten, and batches pending release in a single view.

These ecommerce use cases and their actors are summarized in Fig. 3.

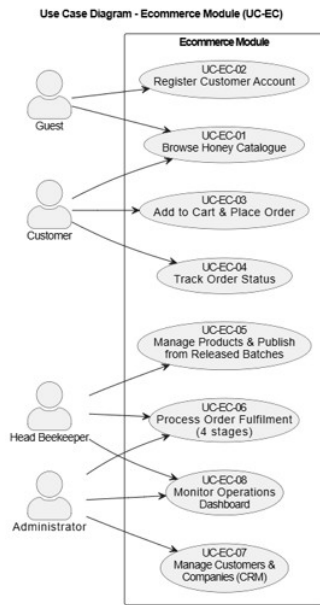


Fig. 3. Ecommerce use case diagram (UC-EC-01-UC-EC-08): catalogue browsing, customer registration, ordering, order tracking, product publication, order fulfilment, customer management, and the operations dashboard.

IV. DESIGN

Honigmanufaktur Lindenhof's management system takes the form of a Monolithic Layered MVC application [8] built on the Yii2 Advanced multi-application template and composed of two independent web applications sharing a common domain layer. PHP 8.3 and the Yii2 framework carry all server-side logic; MariaDB 10.11 persists in the domain data; and Apache 2.4 serves both the backend staff panel and the public frontend on the university virtual machine. This chapter describes how the system is divided into parts and how those parts communicate; the corresponding coding and configuration choices are covered in Chapter V. Fig. 4 illustrates the deployment topology underlying this design: the frontend and backend applications sharing the common domain layer, deployed on the university virtual machine and connected to MariaDB through the lindenhof_app account; the three bounded modules and four infrastructure components that realise the domain layer are described in the remainder of this section.

A. Component Structure

The application follows a Monolithic Layered MVC structure realised through the Yii2 Advanced multi-application template. Three bounded modules form the application core, partitioned across two independent web applications-frontend (public shop) and backend (staff administration panel)-that share the common/ domain layer containing all ActiveRecord models, business logic, and compliance rules [12]. Infrastructure concerns (database, export, HTTP routing) are handled by framework-provided components in the layer beneath the domain models.

The Production Management component captures all raw operational records: the company profile, the Apiary Stand and Colony registries, inspections, treatments, harvests, and batch production details. It is the sole origin of compliance-relevant data; no other component creates or modifies production records. The Compliance component reads production records to evaluate the Release Gate, generate the legally required export documents, manage disease concerns, and support recall traceability. No new data are entered in this component. The Ecommerce component provides the public-facing honey shop, product publication from released batches, order management, customer CRM, and the operations dashboard.

The Data component, implemented through Yii2 ActiveRecord, provides object-relational mapping and keeps all persistence logic outside the domain models. The Export component generates both UTF-8 CSV with byte-order mark and mPDF-rendered PDF documents from the same underlying record set, so that the export format can be changed without modifying the business logic. The HTTP Routing component is provided by the Yii2 MVC layer, which translates incoming HTTP requests into controller actions and translates controller results back into HTTP responses. The Audit component operates at two levels: BlameableBehavior and TimestampBehavior, applied to all domain models, automatically associate every record write with the acting user identity and a Unix timestamp; explicit audit fields on the Batch and Colony models and a dedicated order_stage_log table capture the operator identity and sequence of compliance-critical business events - batch release, disease flag lifecycle, and order fulfilment transitions.

B. Data Model

The persistent data are organized into fourteen domain tables plus a treatment_product approved-product reference table. The colony-centred compliance chain runs from apiary_stand through colony to the inspection and treatment records, then through batch and product to order_item. The batch_colony junction table links each batch to its source colonies, enabling the Release Gate to evaluate withdrawal periods per colony and enabling the recall search to traverse the chain in reverse from a lot number to all affected orders. The colony_movement table records every stand transfer with origin and destination stands, so that treatment records remain associated with the stand at which they were recorded regardless of later colony movements. The treatment_product table holds an approved-product reference used to autofill treatment fields, reducing data-entry time and transcription

errors in the field. The `order_stage_log` table records every fulfilment transition with a timestamp and operator identity, providing an audit trail alongside the order record. Foreign-key constraints enforce referential integrity across the entire chain: a product cannot reference an unreleased batch, and an order line cannot reference a product that does not exist.

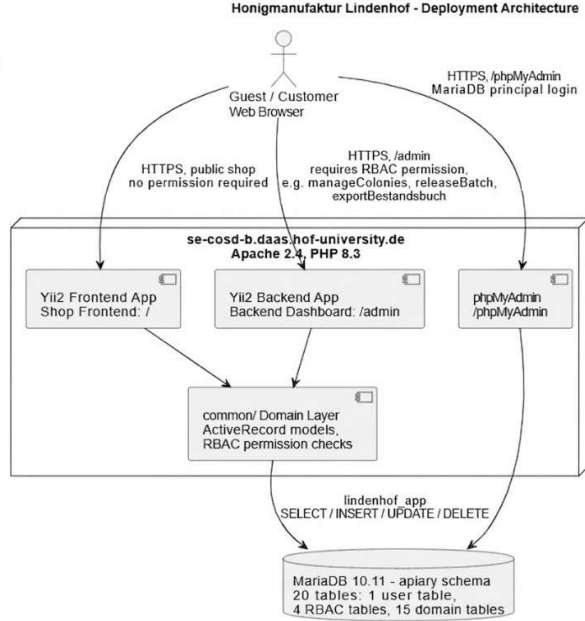


Fig. 4. Lindenhof deployment topology: frontend and backend applications sharing the common domain layer, deployed on the university virtual machine and connected to MariaDB through the `lindenhof_app` account, with distinct HTTP access patterns for each of the five actor roles.

The company profile is a single-row table holding the legal Tierhalter identity required in Bestandsbuch export headers under EU Regulation 2019/6 Article 108(2)(f) [1]. Only the administrator may view and edit this record; its contents are read automatically at export time without requiring manual re-entry.

C. Access Control

Authorization is enforced by Yii2's DbManager-backed RBAC component. Table V presents five structural roles, of which four are backed by explicit database entries seeded in the RBAC migrations: administrator, headBeekeeper, fieldBeekeeper, and customer. The fifth, Guest, is Yii2's built-in unauthenticated marker rather than a seeded RBAC entity; it carries no assigned permissions of its own, and the public actions listed against it in Table V are simply the controller actions that require no permission check at all. Each of the four seeded roles carries a set of named permissions—`manageColonies`, `releaseBatch`, and so on—and every controller action that modifies data demands one of those permissions at its entry point. Decoupling the permission check from the role name means that the role hierarchy can be restructured in the RBAC seed migration without touching a single controller. A request that fails the check receives an explicit HTTP 403 response; there are no silent redirects. The full RBAC scheme is bootstrapped

entirely through versioned migrations, so the permission configuration is reproducible on any clean database. Head Beekeeper and Administrator are granted an identical Compliance and Ecommerce permission set by design - both roles can release batches, run every export and recall-trace query, and monitor the dashboard - so Table V and the use-case diagrams in Section III each show one plausible operator for a given task rather than an exclusive assignment.

TABLE V. SYSTEM ROLES AND RESPONSIBILITIES

Role	Principals	Module Access	Key Permissions
Guest	Unauthenticated visitors (Yii2 built-in marker; not a seeded RBAC entry)	Ecommerce public shop	Browse catalogue, view product detail and lot traceability
Customer	Registered, active buyers	Ecommerce account, cart, orders	Place orders, view own order history and status
Field Beekeeper	Field apiculturalists	Production Management	Log inspections, record treatments, record harvests, view Stockkarte
Head Beekeeper	Production director	Production Mgmt., Compliance, Ecommerce	Release batches, publish products, manage dashboard, flag disease, export Stockkarte
Administrator	Office and logistics staff	All modules	Colony register, user accounts, company profile, customer records, Bestandsbuch export, recall trace

V. IMPLEMENTATION

The running environment is the Hof University shared virtual machine `se-cosd-b.daas.hof-university.de`, where Apache 2.4 routes HTTP traffic to PHP 8.3 entry scripts and MariaDB 10.11 stores all domain data.

A. Migration-Driven Schema

In accordance with NFR2, the entire database schema is defined through Yii2 migrations rather than through manual changes applied via a database GUI [12]. A single `php yii migrate` command from a clean database instantiates twenty tables in total—the base Yii2 user table, four RBAC infrastructure tables (`auth_rule`, `auth_item`, `auth_item_child`, `auth_assignment`) from the `yii/rbac` package, and the fifteen domain tables (fourteen core entities plus the

treatment_product reference table)-seeds the RBAC roles and permissions, and loads demonstration data. The RBAC schema migrations are run first, establishing the role and permission tables as a prerequisite. Application migrations follow in dependency order: company_profile and apiary_stand are created before colony, which is created before inspection, treatment, and batch, which are created before the ecommerce tables. Foreign-key definitions appear within the same migration as the dependent table, so the sequence is self-consistent. Each migration provides both safeUp and safeDown methods. Six additive migrations followed the initial schema, each adding a single column or reference table in response to implementation discoveries, preserving clean-build reproducibility throughout development.

B. Separated Database Users

NFR1 is implemented through three MariaDB principals with strictly separated responsibilities [12]. lindenhof_admin is the schema owner: it holds the ALTER and CREATE rights needed to execute migrations and is never used by the web process. lindenhof_app is the web-process account: INSERT, UPDATE, DELETE, and SELECT on application tables only, with no DDL rights whatsoever. lindenhof_report holds SELECT-only grants and is the account used for Bestandsbuch exports, Stockkarte exports, and recall queries, wired at the connection layer through a dedicated db_report component consumed by ComplianceController and PdfExportService. No application-facing account can promote its own privileges or modify the schema. The account-provisioning migration is Data Control Language rather than schema definition, issued through raw SQL because the Yii2 schema builder does not model user and grant statements; its GRANT statements hardcode the apiary schema name literally rather than reading it from the active database connection, a known constraint if the migration were ever replayed against a dynamically named isolated test database (for example, an apiary_test schema), where the grants would silently apply to the wrong, literally named schema rather than the one under test.

C. Wartezeit Calculation and Release Gate

Withdrawal period enforcement is the most safety-critical operation in the system. When a treatment record is saved, the Treatment model calculates the wartezeit_expiry date by adding the treatment's withdrawal_days value to the application_date using PHP DateTime arithmetic, which correctly handles month-boundary edge cases. The expiry date is stored on the treatment record at creation time and never recalculated, so it reflects the statutory period as it stood at the time of treatment.

The Release Gate is evaluated each time a batch is opened in the Compliance module. The gate checks five conditions in sequence: first, that all source colony Wartezeiten had expired before the harvest date; second, that the batch water content does not exceed the statutory limit for its declared honey variety under HonigV [10]; third, that all mandatory label fields are present; fourth, that no source colony carries an active disease flag; and fifth, that a HACCP process confirmation has been recorded. Condition one is evaluated against the harvest date rather than the current date,

preventing a treatment applied after the harvest from incorrectly blocking a batch whose honey was collected before the treatment was administered. The release action is rendered in the interface only when all five conditions pass simultaneously, and it exists in exactly one place in the application: ComplianceController.

When a colony receives a disease flag after one of its batches has already been released, the system automatically returns that batch to review-required status and unpublishes every product derived from it, leaving the product's stock allocation untouched rather than restoring or reallocating it. The review notes, the original release identity, and the original release timestamp are preserved in the batch record. Re-release is performed manually by the head beekeeper after the concern is resolved, and restores exactly the products that the cascade unpublished, preserving the audit trail of the flag event permanently.

D. Transactional Operations

Three operations write multiple rows and require transactional integrity. Bulk treatment entry wraps all individual treatment inserts in a single database transaction: if any record fails to save, none are written. Customer order placement validates the cart against live stock, reduces each product's stock count, and writes the order and all its lines together so that an order is never left in a partially saved state. Order cancellation restores each line's stock within a transaction and appends an entry to the fulfilment stage log in the same write. Yii2's Transaction component is used throughout, relying on the transactional guarantees provided by MariaDB's InnoDB storage engine.

E. Approved-Product Reference and Autofill

To reduce transcription errors and the time needed to log a treatment from the field (NFR5), a seeded reference table holds common Varroa and Tracheenmilbe medications approved in Germany, each with its default withdrawal period, treatment duration, quantity per colony, and supplier details. When the beekeeper selects a product name in the treatment form, a JSON endpoint returns the product's reference values and a JavaScript handler autofills the corresponding form fields. All autofilled fields remain editable, allowing the beekeeper to record a dosage or batch number that differs from the reference. The logged-in user's display name is inserted into the operator field automatically, and the count of colonies treated at the stand is calculated from the current active colony count at the selected stand, eliminating a manual counting step that was a source of error in the paper-based workflow.

F. Frontend Public Shop

The public-facing shop is implemented as a separate Yii2 frontend application within the advanced-template architecture, sharing the domain models and the database with the backend panel. Guest users may browse the catalogue and read product detail pages, including a lot traceability panel that surfaces the batch lot number, honey variety, harvest date, water content, and origin statement inherited from the linked batch. Registration requires an email address and password; the account and its Customer CRM record are activated immediately on signup. Email verification and password reset are scoped for V2, pending

SMTP provisioning on the university VM. A session-based cart holds items across page loads. At checkout the system validates the delivery address, checks live stock for each cart line, decrements stock transactionally on confirmation, and snapshots the batch lot number onto each order line item. Wholesale customers are identified by a flag on the customer record; if a wholesale price is set on a product, it is applied at checkout in place of the retail price, and a minimum order quantity is enforced before the order is accepted.

G. Version Control

GitLab serves as the single source of truth for the project. The team adopted a two-branch strategy aligned with the development lifecycle. The main/master branch retains the stable Yii2 Advanced Project Template skeleton, ensuring a pristine baseline for core application configurations. The developer branch functions as the integration branch where team members commit operational features, custom layouts, business logic, and database migrations. Developers clone the central repository, implement features on their local checkout, and promote code to the live instance only after peer review-mirroring the branch-and-promote discipline described above.

Database schema consistency across separate development environments is enforced through migration-tracked PHP blueprints rather than SQL dump transfers. When a team member commits a new migration (for example, the RBAC seed `m260615_000001_rbac_seed.php`), other developers receive it on git pull and apply it locally via `php yii migrate`-ensuring byte-for-byte schema equivalence across all machines. Environment separation is maintained through a `.gitignore` strategy: shared structural configurations (`main.php`) are versioned and pushed to GitLab, while local instance-specific files (`main-local.php`, `codeception-local.php`) are excluded. After cloning, developers run `php init` to generate local configuration stubs, preventing credential leakage to the remote repository.

The repository also coordinates the Codeception test environment. Because auto-generated test helper classes (such as `UnitTester`) are excluded from version storage to avoid repository bloat, developers run `vendor/bin/codecept build` after each pull to rebuild them from the fetched branch configuration, ensuring the test runner integrates correctly without missing-file exceptions.

H. Deployment Environment and Access Matrix

The deployed system is accessible via the Hof University shared virtual machine. Table VI lists the available web interfaces and their corresponding URLs. Table VII provides the login credentials for each role defined in the RBAC model. The public shop and the staff backend are the same Yii2 Advanced Template deployment described in Sections V.A-B, distinguished only by URL path and by the RBAC permissions enforced on each controller action; `phpMyAdmin` is a separate database-administration tool exposed alongside the application for schema inspection and authenticates against the MariaDB principals of Section V.B rather than against the application's own RBAC accounts. Five demonstration application accounts are seeded across the four RBAC roles for grading and demonstration purposes, covering both the retail and wholesale ecommerce paths; all

five share a single demonstration password, and none of the three MariaDB principals share credentials with any application account.

TABLE VI. WEB INTERFACE URLS

Interface	URL
Shop Frontend	http://se-cosd-b.daas.hof-university.de/
Backend Dashboard	http://se-cosd-b.daas.hof-university.de/admin
phpMyAdmin	http://se-cosd-b.daas.hof-university.de/phpMyAdmin

TABLE VII. USER CREDENTIALS BY ROLES

Role	Username(s)	Password
Administrator	admin	COSD@groupb
Field Beekeeper	fieldBeekeeper	COSD@groupb
Head Beekeeper	headBeekeeper	COSD@groupb
Customer	customer	COSD@groupb
Wholesale Customer	wholeSaleCustomer	COSD@groupb

VI. VERIFICATION

Emil Shkurti, as the designated testing specialist, owns the test suite, its documentation, and all coverage decisions in this chapter [12]. The project uses Codeception [11], bundled with the Yii2 advanced template, running one functional suite against the backend application and one against the frontend. Every journey deliberately pairs a success case with at least one failure case, because a suite that only walks the happy path cannot prove that invalid input is correctly rejected [9]. A browser-driven suite was explored but ruled out: the university virtual machine cannot resolve its own public DNS name through the campus network proxy, making Selenium-style HTTP round-trips impossible; all tests therefore drive the Yii2 request pipeline in-process. The Yii2 Advanced Template's default Acceptance suite (HomeCest, exercising unmodified template boilerplate rather than Lindenhof content) remains present in the codebase tree and is not removed; it is not exercised as part of the results reported in this chapter, but is retained unmodified as a baseline environmental smoke test should browser automation become viable in a future deployment.

A. Test Setup

Codeception initialises a dedicated `apiary_test` schema by replaying the full migration sequence-RBAC tables first, then domain tables-before each suite run, so the structure under test is byte-for-byte identical to what is deployed in production. Write operations inside each individual test execute within a savepoint that the DbCleaner module rolls back on teardown; no residual rows accumulate between cases, and the suite can be repeated without manual schema resets. Each journey seeds only the rows it needs through explicit fixture calls.

B. Journey 1 - Harvest to Release

The backend functional suite contains ten tests in the `HarvestToReleaseCest` class, covering 76 assertions. The journey walks the complete Production Management to Compliance workflow. The first two tests establish preconditions: `adminCreatesApiaryStand` registers an apiary stand, and `adminRegistersColonyAtStand` links a colony to it. `treatmentRecordedWithWartezeitCalculated` verifies that saving a treatment with `application_date` 2026-06-01 and a 21-day withdrawal period creates a record with `wartezeit_expiry` 2026-06-22 computed by PHP `DateTime` arithmetic. `harvestCreatesBatchInPendingRelease` confirms that a harvest event creates batch LIN-2026-004 in pending-release status with the correct source colony associations.

`releaseGateBlockedByActiveWartezeit` is a deliberate failure case: it attempts to evaluate the gate while a source colony is within its withdrawal period and asserts that condition one returns a failing result identifying the colony. `releaseSucceedsWhenAllChecksPassed` confirms the full gate evaluation when all five conditions are met and asserts that the batch transitions to released status with a timestamped release event. `diseaseFlagPutsReleasedBatchUnderReviewAndUnpublishesProduct` verifies the cascade: setting a disease flag on a source colony of a released batch moves the batch to review-required and unpublishes the derived product. The final three tests cross the Compliance-to-Ecommerce boundary: `headBeekeeperCreatesProductFromReleasedBatch` confirms provenance inheritance from the batch; `bestandsbuchExportProducesCSVForStand` confirms the CSV export contains all EU Regulation 2019/6 Article 108(2) mandatory fields [1]; and `recallTraceFindsOrdersForLotNumber` confirms that a lot-number search for lot LIN-2026-003 returns all affected orders in the chain-surfacing order ORD-2026-9001 for customer Hans Müller, including delivered orders.

C. Journey 2 - Shop Checkout

The frontend functional suite contains four tests in the `ShopCheckoutCest` class. `productDetailShowsLotTraceability` confirms that the product detail page surfaces the batch lot number, honey variety, harvest date, water content, and origin statement from the linked batch record. `checkoutDecrementsStockAndSnapshotsLotNumber` verifies that a successful 3-unit checkout reduces product stock from 10 to 7 and snapshots the source batch lot number onto the order line item, confirmed by direct model assertions on the `Order` and `OrderItem` records. `checkoutRequiresDeliveryAddress` is a failure case: it submits the checkout form with the address field empty and asserts that the order is rejected. `wholesaleMinimumEnforced` confirms that a wholesale customer attempting to order below the minimum quantity is blocked at checkout.

D. Template Test Adaptation

The Yii2 Advanced Template ships with functional test classes covering authentication, form validation, and email verification flows. During development these classes were updated to match Lindenhof's routes, page content, and modified application behaviour: `HomeCest` asserts against

Honigmanufaktur Lindenhof content; `LoginCest` verifies the role-aware post-login redirect to the Dashboard; and `SignupCest` confirms immediate account activation without email confirmation. The adapted suite was retained because the authentication and input-validation flows it exercises underpin the domain journeys; the classes were updated as a maintenance obligation, not a deliberate extension of domain coverage.

E. Test Results

Thirty-nine tests pass in total: ten from `HarvestToReleaseCest`, four from `ShopCheckoutCest`, and twenty-five from the adapted Yii2 template suite. The primary verification of domain logic rests on the fourteen journey tests; the remaining twenty-five confirm that authentication and form validation continue to function correctly after Lindenhof's modifications, alongside the legacy email-verification test classes retained from the template, ready for v2, but not exercised by the live registration flow. Including failure cases across both journey suites is deliberate: the withdrawal enforcement and Release Gate requirements demand that the system reject invalid input, and confirming only the happy path would not demonstrate that behaviour.

VII. VALIDATION

Where Chapter VI established that each component behaves as its specification demands, this chapter asks the harder question: does the assembled system actually serve a Bavarian honey producer operating under EU veterinary pharmaceutical law [12]? Every functional requirement from Chapter III is traced to observable behaviour in the running application, with the three highest-risk areas—Wartezeit enforcement, the five-condition Release Gate, and per-order lot traceability—receiving the most detailed scrutiny.

A. End-to-End User Journeys

Three journeys were conducted against the live instance to confirm correct behaviour across module boundaries.

Field beekeeper journey. A field beekeeper logs in, navigates to the stand view, and selects three colonies for a bulk Varroa treatment. The treatment form is pre-filled from the approved-product reference with withdrawal period, treatment duration, quantity per colony, and supplier details. After saving, each colony's `Stockkarte` shows the computed Wartezeit expiry date, and all three colonies appear on the dashboard panel for colonies within an active Wartezeit. The beekeeper then attempts to link two of the treated colonies to a harvest event dated within the withdrawal period; the system rejects the submission and identifies the blocking treatment on each colony by name.

Compliance and release journey. The head beekeeper opens a pending batch whose source colonies are past their Wartezeit. The Release Gate shows all five conditions passing. The beekeeper releases the batch; the release event is timestamped with the beekeeper's identity. The administrator generates the `Bestandsbuch` export for all stands as both CSV and PDF. The PDF header carries the Tierhalter name and address from the company profile; every data row contains all eleven Article 108(2) mandatory fields [1]. A second test sets a disease flag

on a source colony of a different released batch; the system immediately marks that batch as requiring review and unpublishes the derived product, leaving its stock allocation untouched. After the flag is cleared, the head beekeeper re-releases the batch through the Release Gate; the products are republished and the original review note is preserved in the batch audit trail.

Ecommerce and traceability journey. A guest browses the catalogue, registers a customer account, adds two products to the cart, and completes checkout with a valid delivery address. The order confirmation records the batch lot number on each order line. A staff member advances the order through the four fulfilment stages. The administrator runs a recall search by the batch lot number and retrieves the order immediately, including the delivery address, customer name, and quantity ordered, confirming end-to-end traceability from the customer order back to the source colony.

B. Requirements Coverage

All 23 functional requirements and all eight non-functional requirements from Chapter III are met in the running system. FR9, which specifies the Release Gate, and FR6, which specifies Wartezeit enforcement, carry the highest regulatory risk; both are covered by automated tests in Chapter VI and by the compliance and release journey above. FR15, which specifies lot number recording on order lines, is confirmed by the ecommerce and traceability journey. NFR3, which requires auditability, is confirmed by the presence of creator identity and timestamp on every compliance-relevant record, verified by direct database inspection during the end-to-end journeys described above.

VIII. PROJECT MANAGEMENT

The project was planned and tracked in Jira using a Scrum template across four one-week sprints (18 May-15 June 2026), with a fifth buffer week reserved for integration testing and documentation. The backlog comprised 12 user stories organized into three epics: Production Management (LH-01-LH-05, 41 SP), Compliance (LH-06-LH-08, 28 SP), and Ecommerce (LH-09-LH-12, 29 SP), totaling 98 story points across the team.

A. Sprint Execution and Velocity

Table VIII summarises sprint outcomes. Average velocity was 24.5 SP per sprint. Sprint 2 delivered 23 SP against a plan of 26: US-10 (order checkout) was blocked by Wartezeit gate logic from US-03, which itself slipped one day due to a transactional refactor required for all-or-nothing semantics on bulk treatment entry. Sprint 3 recovered the 3-SP deficit and over-delivered (30 SP against a plan of 27). The remaining two sprints closed on plan.

TABLE VIII. SPRINT SUMMARY

Sprint	Dates	Focus	Plan SP	Done SP	Delta	Status
1	18-25 May	System setup, apiary registry, colony register, inspections, public catalogue	24	24	0	On Time

2	26 May-1 Jun	Veterinary treatments, harvest & batch production, order placement	26	23	-3	Minor Delay
3	2-8 Jun	Compliance gate, Bestandsbuch/Stockkarte exports, storefront fulfilment	27	30	+3	Ahead of Plan
4	9-15 Jun	Compliance exports, CRM, operations dashboard, RBAC hardening	21	21	0	On Time
Total		98 SP delivered across 4 sprints (100%)	98	98	0	

B. Bug Tracking

Three bugs were identified and resolved within the sprint in which they were found, each logged as a Jira issue linked to its parent story and verified fixed before sprint close. LH-B1 (High, 2 SP, Sprint 2): Wartezeit not blocking harvest on exact expiry date-a strict less-than operator was corrected to less-than-or-equal in the date comparison. LH-B2 (Critical, 2 SP, Sprint 3): Disease flag cascade missing unpublish step after a batch had already been released-the event listener was extended to cover the post-release scenario. LH-B3 (Medium, 1 SP, Sprint 4): Off-by-one in container yield calculation-floor() was applied to the integer division in the yield formula. No story was affected beyond its owning sprint.

C. Milestones and Outcome

Six milestones structured delivery. M1 (Kickoff, 18 May): Jira configured and backlog groomed. M2 (Data Layer Live, 25 May): registry, colonies, and inspections deployed. M3 (Field Ops Done, target 1 Jun, actual 3 Jun): treatments, harvest-to-batch, and Stockkarte-two days late due to the US-03 refactor and the US-10 rollover. M4 (Compliance Live, 8 Jun): release gate, disease cascade, and recall search. M5 (Feature Complete, 15 Jun): all 12 stories Done, RBAC hardened, 103 acceptance criteria verified at 100% pass rate. M6 (Demo/Submit, 22 Jun): live demonstration on the university server and project closure. The final Jira board state shows all issues as Done with 0 items remaining in any active state.

D. Team Workload Distribution

Emil Shkurti delivered 36 SP (including 2 SP for bug LH-B2), leading the compliance gate logic and ecommerce checkout flow. Hafsa Iddris also delivered 36 SP (including 1 SP for bug LH-B3), owning entity registration, batch production, and CRM. Fahimeh delivered 26 SP (including 2 SP for bug LH-B1), responsible for the treatment module, compliance exports, and Scrum facilitation. All three team members verified each other's acceptance criteria walkthroughs in their assigned story roles before sprint close.

IX. FUTURE THINGS / RESUMÉE

A. Version 2 Scope

Six requirements were fully specified but deferred to a planned Version 2. They extend the Compliance module into active regulatory reporting rather than passive record-keeping. The first requirement would record and track authority notifications to the Veterinäramt for colony registration, relocation, and disease outbreak, including the Behörden-Registernummer returned by the authority under §1a BienSeuchV. The second would enforce a health certificate requirement when colonies move across Veterinäramt jurisdictions. The third would manage declared disease cases with stand lockdowns and Sperrbezirk radius declarations under §§3-12 BienSeuchV. The fourth would calculate legal colony-count threshold alerts under German insurance, agricultural, and tax regulations. The fifth would manage pesticide consent requests from neighbouring farmers for the 60-metre colony protection zone. The sixth would enforce the complete HonigV label field set on every batch, including HMF value, conductivity, and sugar content, blocking publication until all fields are present and within statutory limits. The existing data model carries placeholder columns for all Version 2 fields, so each extension can be added without structural change to the shared schema. A structured logging adapter for event streaming to an external service is identified as a further Version 2 extension point, building on the audit foundation already established through BlameableBehavior and the order_stage_log table in the current implementation. Customer-facing notification is deferred for the same reason: the Yii2 mail component is configured but not exercised by any current flow, since no SMTP service was provisioned on the university VM; order-confirmation and account-verification email delivery are scoped for Version 2, when the component can be pointed at an external delivery service without altering the ordering or signup flow.

B. Lessons Learned

Three lessons from this project merit recording. First, specifying the Release Gate as an automatically evaluated checklist rather than a manual confirmation proved important early in development: a manual checkbox would require the head beekeeper to recall all five conditions independently, recreating the paper-based compliance risk in digital form. Automating the gate from existing records eliminates the memory dependency entirely. Second, isolating the Wartezeit calculation to a single point in the Treatment model and storing the result rather than recalculating on demand proved critical for correctness: recalculating at export time could produce a different result if the calculation logic were later revised, breaking the immutability of a statutory record. Third, ordering migrations carefully by foreign-key dependency-and providing both up and down methods on each-made clean-build reproducibility reliable from the first day of integration work, which in turn made the Codeception test setup straightforward.

C. Conclusion

The Lindenhof platform replaces a paper-based, legally fragile workflow with a single cohesive data loop: a treatment entered in the field automatically computes the Wartezeit,

gates batch release at the facility, and gates product publication in the shop, ensuring that every customer order permanently carries a traceable lot reference back to its source colony. The Monolithic Layered MVC design keeps module boundaries explicit, with all business and compliance logic encapsulated in the shared common domain layer and infrastructure concerns delegated to the Yii2 framework, so that either application tier can evolve without disturbing the domain models. The migration-driven schema and three separated database users mean the system can be reproduced, audited, and maintained without dependency on individual knowledge or manual database intervention.

X. APPENDIX: AI USAGE DOCUMENTATION

This appendix documents each significant AI interaction during the project, following the team's agreed eight-field template. For each entry, the owner is the team member who made the final technical and correctness decisions. The AI generated code or analysis, or IEEE document content within boundaries the owner set; the owner then verified and accepted or revised the output. Fields per entry: Context, Role, Boundaries, Goals, Voice, Iterations, Verification, Team Decision.

AI-01 - Production Management Module | Owner: Emil

Context: No codebase existed; Production Management module needed from scratch against the supplied requirements.

Role: Generated models, migrations, controllers, and views for CompanyProfile, ApiaryStand, Colony, ColonyMovement, Inspection, Treatment, and Batch.

Boundaries: RBAC permission-not-role checks on every action; strict_types and return type declarations throughout; controllers thin - business logic in models only.

Goals: Working Production Management module with domain schema and full CRUD behind a dark-themed backend layout.

Voice: Senior Yii2 backend developer; working PHP code only - no explanatory prose between code blocks.

Iterations: 2 - repeated because the first output failed manual testing: login CSRF fix on Windows (cookie path); form card contrast correction.

Verification: Emil manually tested all CRUD flows and role-based access.

Team Decision: strict_types and return type declarations adopted as team baseline for all subsequent work.

AI-02 - Compliance Module | Owner: Hafsa

Context: Production Management complete; Compliance module needed against EU Reg. 2019/6 Art. 108(2), with every compliance read served through a dedicated least-privilege database account per NFR1.

Role: Generated ComplianceController and views for Release Gate, Bestandsbuch CSV export, Stockkarte export, and Recall Trace, and provisioned the three-tier MariaDB account model (lindenhof_admin, lindenhof_app, lindenhof_report) that keeps compliance reads on a SELECT-only connection.

Boundaries: Call `Batch::getReleaseGateChecks()` - do not rewrite release logic; Bestandsbuch columns must match Art. 108(2) exactly; use the existing backend layout; `linden_hof_report` is SELECT-only and is the only account the export and recall-trace methods read through; account privileges are provisioned via a raw-SQL migration since account management is DCL and falls outside the Yii schema builder.

Goals: Four working compliance pages with correct exports and a release gate displaying all five checks, with every export and recall-trace read routed through the read-only reporting account.

Voice: Senior Yii2 backend developer with EU regulatory domain knowledge; working PHP code only - EU legal terms preserved as-is.

Iterations: 4 - repeated because the initial version misbehaved at runtime: RBAC redirect fix (exports going to home); dark-background date picker visibility fix; a fourth pass provisioned the three MariaDB accounts and wired the reporting connection into every export and recall-trace path.

Verification: Emil tested all four pages, verified Bestandsbuch column headers against the regulation text, and confirmed every export and recall-trace read executes through the `linden_hof_report` connection.

Team Decision: Disease flag management stays in Colony views - AI proposed a dedicated compliance sub-page; team overruled.

AI-03 - Ecommerce Backend | Owner: Fahimeh

Context: Backend complete; Ecommerce module needed - Products, Orders, Customers, Dashboard.

Role: Generated `DashboardController`, `ProductController`, `OrderController` (with Kanban view grouping orders by fulfilment stage), `CustomerController`, and all views.

Boundaries: Only released batches may source a product; product inherits lot number, variety, harvest date, origin from batch (read-only); dashboard reads live from database; dashboard is the default landing page.

Goals: Complete Ecommerce backend with working Kanban, customer CRM, and live dashboard.

Voice: Senior Yii2 full-stack developer; working PHP code only - no explanations, consistent with established backend conventions.

Iterations: 1 - one revision needed because the AI leaked an internal requirement ID (AC-EC-05.1) into a UI label; removed.

Verification: Emil tested Kanban stage transitions, confirmed KPIs read live data, verified batch dropdown shows released batches only.

Team Decision: Dashboard set as `defaultRoute` - AI suggested keeping `site/index`; team overruled.

AI-04 - Frontend Public Shop | Owner: Emil

Context: Backend complete across all modules; public-facing frontend shop needed building.

Role: Generated `ShopController`, `CartController`, `CheckoutController`, `AccountController`, and all views with a warm honey-themed design.

Boundaries: Cart session-based; checkout must snapshot `lot_number` and `batch_id` on every order item; wholesale minimum enforced at checkout; amber/gold aesthetic.

Goals: Public shop where registered customers can purchase and every order line carries the source batch lot number.

Voice: Senior Yii2 frontend developer; working PHP code and views only - warm amber/honey-brand aesthetic throughout.

Iterations: 2 - repeated to correct visual defects and remove unnecessary AI-generated content: styling fixes (login contrast, order detail visibility); removed placeholder pages.

Verification: Emil tested full flow: browse → register → cart → checkout → confirmed lot number on order items in the database.

Team Decision: Sold-out products stay published - AI initially hid them; team required the lot traceability page to remain publicly accessible for recall.

AI-05 - UX Gap Closure | Owner: Fahimeh

Context: Manual testing revealed UX gaps - treatment form required manual data entry, honey variety was free text, feeding data untracked, wholesale pricing missing.

Role: Added `treatment_product` reference table seeded with real approved German Varroa products, treatment form AJAX autofill, honey variety constrained to dropdown, wholesale price field, and feeding fields on inspections.

Boundaries: Seed real products (Oxuvor, Apivar, MAQS, Thymovar) with autofill for `Wartezeit` days, duration, quantity, supplier; honey variety as constrained dropdown to enable HonigV water-content validation.

Goals: Treatment form that autofills from a seeded product reference with all UX gaps closed.

Voice: Senior Yii2 developer; implement changes in specified order - working code only, no unrelated modifications.

Iterations: 2 - split deliberately into paired sessions to keep each scope small and verifiable (backend then frontend and release gate navigation links), not due to errors.

Verification: Emil tested Oxuvor autofill and confirmed `Wartezeit` days populated correctly.

Team Decision: `treatment_product` is a seeded reference table of real approved products - team decision to minimise field data-entry errors for the Bestandsbuch.

AI-06 - Critical Fixes and Architectural Cleanup | Owner: Hafsa

Context: End-to-end audit identified eight functional gaps and a structural duplication - disease flag cascade missing, bulk treatment not transactional, order cancellation not restoring stock, lot number mutable, Customer record created lazily, and the release action duplicated across `BatchController` and `ComplianceController`.

Role: Fixed all eight functional gaps in order, then removed `BatchController::actionRelease` and updated the `Batch` view to show a read-only gate summary with a link to Compliance only.

Boundaries: Disease flag must automatically set released batches to `review_required` and unpublish products; bulk treatment must use a database transaction; lot number

immutable via beforeSave guard; only ComplianceController may release a batch - Production Management records, Compliance decides.

Goals: All gaps resolved and a single unambiguous release authority established.

Voice: Senior Yii2 developer applying targeted fixes; test each fix before proceeding to the next - working code only.

Iterations: 3 - two repeats because generated fixes were faulty: disease flag button JavaScript not responding; review_required state not clearing after flag resolved; plus a prior analysis session to confirm the architectural fix before any code changed.

Verification: Emil tested the full disease flag lifecycle: set → review_required → products unpublished → clear → re-release → products restored → review_note preserved. Confirmed release unreachable from any Production Management page.

Team Decision: Clearing the disease flag does not auto-revert the batch - manual re-release required to preserve audit trail. Withdrawal check evaluates against harvest_date, not today - post-harvest treatments cannot contaminate already-extracted honey.

AI-07 - Batch-Product Flow and PDF Export | Owner: Fahimeh

Context: Batch-to-product flow needed allocation tracking; Bestandsbuch needed an all-stands export option; PDF export needed mPDF reproducing the official LWG Bayern templates.

Role: Implemented batch allocation panel, unified product creation form, all-stands and all-colonies export options, and PdfExportService using mPDF for Bestandsbuch (landscape A4) and Stockkarte (portrait A4).

Boundaries: Sold-out products stay published; product creation unified - same form from batch view and Products → New; Release Gate is a worklist (pending_release and review_required only); PDF must reproduce the official LWG Bayern column layout.

Goals: Complete batch-to-product pipeline with PDF export matching the LWG Bayern paper template.

Voice: Senior Yii2 developer and mPDF specialist; working PHP code only - PDF output must visually reproduce the official paper template.

Iterations: Multiple - repeated because output violated the set boundaries: batch view showing compliance data it should not; create product button appearing with zero units remaining; plus a mid-session recovery after dropped connection.

Verification: Emil tested product creation from both entry points and confirmed PDF columns matched Art. 108(2).

Team Decision: Batch-to-product is a manual step - Lindenhof splits one batch into multiple products at different prices; AI suggested auto-create on release; team rejected it.

AI-08 - Test Suite Design and Implementation | Owner: Emil

Context: No tests existed; paper referenced test files that did not exist; professor requires Codeception tests that read like user journeys with failure cases.

Role: Designed a two-journey test suite covering all three modules, then implemented HarvestToReleaseCest (10 backend functional tests) and ShopCheckoutCest (4 frontend functional tests) with all fixture data.

Boundaries: Codeception only; no Acceptance suite (VM VPN restriction); tests must read like user journeys; backend 10 tests in HarvestToReleaseCest, frontend 4 tests in ShopCheckoutCest; fix all errors before reporting.

Goals: 14 passing tests covering the full Production → Compliance → Ecommerce chain.

Voice: Design phase - software testing consultant; structured options in prose with trade-offs and a clear recommendation. Implementation phase - senior Codeception developer; working PHP test classes only.

Iterations: Design — 3 rounds because the first two proposals did not meet requirements (7-class proposal rejected; module-by-module rejected; final user-journey design accepted). Implementation — 3 fixes caused by environment and data gaps, not logic errors (test database missing; demo seed data polluting order tables; Batch fixture missing packaged_unit_count).

Verification: Emil ran both suites - 10/10 backend (76 assertions), 4/4 frontend - 14/14 green.

Team Decision: Emil chose 14 tests across 2 classes; Recall Trace added to close the traceability chain; pre-existing Yii2 template tests left in place - combined total 39 passing tests.

AI-09 - RBAC Completion and Demo Users | Owner: Hafsa

Context: RBAC was implemented but three gaps remained - no demo users, post-login redirect always went to /dashboard/index (causing 403 for fieldBeekeeper), Administrator role semantics undocumented.

Role: Analysed role semantics against professor transcripts; created the seed migration and fixed the role-aware redirect.

Boundaries: Guest is Yii2's built-in ? marker - no migration needed; do not redefine existing roles; passwords COSD@groupb for all demo users; Customer CRM record must be created in the migration for the customer demo user.

Goals: Three seeded demo users with correct RBAC roles and role-aware post-login redirect.

Voice: Senior Yii2 developer; precise migration and redirect fix only - no unrelated changes.

Iterations: 1 - single pass; no repetition needed — migration applied cleanly on first run.

Verification: Emil verified headBeekeeper → /dashboard/index, fieldBeekeeper → /colony/index, customer → /shop/index, Customer CRM record created correctly.

Team Decision: Administrator role confirmed as Betriebsinhaber (legally responsible Tierhalter), not an IT administrator; UserController omitted deliberately - professor's RBAC minimum is satisfied by enforcement, not a user management UI.

AI-10 - Frontend Polish: Impressum, About Page, Signup Flow | Owner: Fahimeh

Context: Frontend lacked a legally required Impressum (§5 TMG); About page was a placeholder stub; signup flow was

broken - email verification blocked registration and no Customer record was created on signup.

Role: Added Impressum page, rewrote About page with accurate company content, wired footer links on every page, fixed signup flow to activate accounts immediately and create the Customer record at signup.

Boundaries: Impressum linked from footer on every page; email verification deferred to V2 (no SMTP on server); password reset deferred to V2; all About page body text at the same font size; content accurate to paper (Landkreis Hof, not Frankwald).

Goals: Legal compliance page on every page, accurate About page, and working customer registration creating a Customer CRM record immediately.

Voice: Senior Yii2 frontend developer; working PHP code only - match existing shop aesthetic, correct to English where needed.

Iterations: 4 - repeated because the AI's output was wrong each round: wrong file edited (_footer.php instead of main.php); CSS override hiding footer links; About page written in German then corrected to English; Frankwald corrected to Landkreis Hof.

Verification: Frontend suite ran - 28 tests, 62 assertions, all green; Impressum confirmed visible on every page; signup tested end to end.

Team Decision: Email verification and password reset both deferred to V2; About page in English; Impressum stays in German (legal requirement).

AI-11 - IEEE Paper Documentation | Owner: Emil

Context: Sprint logs, module writeups, prior AI-usage session logs, and the IEEE rules existed, but no single document did; the paper had to be built from scratch into a submittable eleven-section paper with abstract, bibliography, and appendix.

Role: Drafted and formatted the whole document - turned notes and session logs into IEEE prose per section, applied two-column structure, headings, citations, and bibliography formatting; generated this appendix (AI-01-AI-10) from chat history. Did not decide what the paper claimed, only how it read.

Boundaries: No invented claims or figures - every sentence traceable to the codebase and project materials; fixed IEEE structure and citation style; existing prose tightened, not rewritten; no unbuilt capability described.

Goals: A complete, submittable IEEE paper reading as one coherent document, matching the delivered code and semester requirements, with an honest AI-usage appendix.

Voice: Academic technical writer; IEEE conference register - plain, factual, third-person, no marketing language.

Iterations: 8 - repeated mainly to correct factual inaccuracies against project records and to tighten and shorten the text: initial draft assembly from notes; fact-correction rounds; final rounds for formatting and cross-section wording.

Verification: Emil checked every section against the system, and project materials; verified IEEE formatting; reviewed this appendix against the session history it was built from.

Team Decision: Several AI-drafted claims and figures were corrected to match project records rather than accepted as-is - Emil, who owns this section, approved the final wording.

AI-12 - Architecture and Use-Case Diagrams | Owner: Hafsa

Context: Ten modules complete, paper drafted; four diagrams still missing - one architecture diagram and three use-case diagrams for Production Management, Compliance, and Ecommerce.

Role: Suggested structure and content for all four diagrams in the team's diagramming tool - box groupings, actor placement, flow arrows. Did not finalize layout; Hafsa built and corrected each one.

Boundaries: Every box, actor, and arrow limited to real components, controllers, and use cases - no aspirational or future-state elements; architecture diagram limited to the actual MVC layers and shared domain layer; use-case actors limited to the three real roles per module.

Goals: Four readable, high-level diagrams - one architecture diagram and three use-case diagrams - accurate to the running system without exhaustive detail.

Voice: Diagram designer; simplified, high-level style - few boxes, clear labels matching real component and use-case names, no field- or method-level detail.

Iterations: 3 - repeated because early drafts were inaccurate to the real system: first architecture pass missing the shared domain layer between the two application tiers; first use-case pass with actors grouped incorrectly; final round fixing flow arrows to match actual controller logic.

Verification: Hafsa checked the architecture diagram against the real MVC folder structure and shared domain layer; checked each use-case diagram against its module's actors, controllers, and the paper's use-case list (UC-PM, UC-CO, UC-EC).

Team Decision: Diagrams kept high-level and simplified rather than exhaustive - Hafsa chose readability at a glance over showing every field or method, leaving out anything not essential to the flow.

XI. BIBLIOGRAPHY

- [1] European Parliament and Council, "Regulation (EU) 2019/6 on veterinary medicinal products," Official Journal of the European Union, Jan. 2019.
- [2] German Federal Government, "Tierarzneimittelgesetz (TAMG 2022): Gesetz über den Verkehr mit Tierarzneimitteln und zur Änderung weiterer Vorschriften," Bundesgesetzblatt, 2021.
- [3] Deutscher Imkerbund e.V., "Statistik: Imker und Völker in Deutschland," Wachtberg, Germany, 2024. [Online]. Available: <https://deutscherimkerbund.de/>
- [4] Bundesministerium für Ernährung und Landwirtschaft (BMEL), "Honig: Erzeugung und Verbrauch," Berlin, Germany, 2024. [Online]. Available: <https://www.bmel.de>

- [5] Bund Ökologische Lebensmittelwirtschaft (BÖLW), “Zahlen, Daten, Fakten: Die Bio-Branche 2025,” Berlin, Germany, 2025. [Online]. Available: <https://www.boelw.de>
- [6] Yiiisoft, “Yii 2 Application Structure - Advanced Project Template,” Yii Framework Documentation. [Online]. Available: <https://www.yiiframework.com/doc/guide/2.0/en/tutorial-advanced-app>
- [7] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-based access control models,” IEEE Computer, vol. 29, no. 2, pp. 38-47, Feb. 1996.
- [8] Yii Software, “The definitive guide to Yii 2.0.” [Online]. Available: <https://www.yiiframework.com/doc/guide/2.0/en>
- [9] I. Sommerville, Software Engineering, 10th ed. Harlow, U.K.: Pearson, 2015.
- [10] German Federal Government, “Verordnung über Honig (Honigverordnung - HonigV),” Bundesgesetzblatt, 2004, last amended 2021.
- [11] Codeception, “Codeception PHP testing framework.” [Online]. Available: <https://codeception.com>
- [12] J. Heym, “Lecture slides: Component-oriented software development,” Hof Univ. of Applied Sciences, Hof, Germany, private communication, 2026